Autonomous Systems
Assignment 03

Argument Lab

Exact Abstract Argumentation and
Strategic Argument Revelation

A project report

Abstract

This report describes *Argument Lab*, a local application for abstract argumentation and for the strategic question of whether an agent can profit from hiding arguments. The system implements Dung's abstract argumentation frameworks with exact enumeration of the grounded, complete, preferred, stable and semi-stable semantics, legal labellings and acceptance status. On top of that it implements the grounded direct mechanism of Chapter 5 of *Multiagent Systems* (Weiss, 2013): best responses, an exhaustive check of strategyproofness (Theorem 5.3) and a separate check of the structural sufficient condition (Theorem 5.4). We validate the implementation against the textbook examples and against independent oracles over all 512 directed three-node graphs, and we run a seeded experiment of 5,000 random three-agent instances. Of these, 1,054 are strategyproof; the fraction falls from 83.7% at attack probability $p = 0.1$ to 0.1% at $p = 0.9$. The structural condition held in 285 instances and *every* one of them was strategyproof, while 769 further instances were strategyproof although the condition failed. An independent audit confirmed all 5,000 decisions.

Name	**Roll number**
Jatin Purbia	B23CH1022
Sharana Gouda Lakshetty	B23CH1042

November 11, 2026

Contents

1 Introduction **2**

2 Background **2**

2.1 Abstract argumentation frameworks 2

2.2 Labellings and acceptance status 3

2.3 Strategic revelation 3

3 Algorithms **3**

3.1 Exact enumeration of semantics 3

3.2 Best responses and the strategyproofness check 4

3.3 The odd-walk condition 4

4 System design **4**

4.1 Architecture 4

4.2 HTTP interface 5

4.3 User interface 6

5 Validation on the textbook examples **6**

6 Experimental method **7**

7 Results **8**

7.1 Strategyproofness against attack density 8

7.2 The structural condition versus measured strategyproofness 9

7.3 Independent audit 10

8 Testing **11**

9 Limitations and future work **11**

10 Conclusion **11**

1 Introduction

Argumentation is a way of reasoning in which conclusions are justified by arguments that can attack one another. In *abstract* argumentation the content of an argument is ignored: only the attack relation between arguments matters. This is enough to define precisely which sets of arguments can be accepted together, and different definitions of "acceptable" give different *semantics*.

When the arguments are put forward by self-interested agents, a second question appears. An agent decides which of its arguments to reveal, and revealing more arguments can change the outcome in ways that hurt the agent itself. A mechanism is *strategyproof* when revealing everything is always at least as good as any other report, whatever the other agents do.

The goals of this project are:

1. implement abstract argumentation frameworks and the five standard semantics exactly;
2. implement the grounded direct mechanism, its best responses, and checks of Theorems 5.3 and 5.4;
3. reproduce the textbook examples, including two textbook errata;
4. run a reproducible random experiment and relate the structural condition to measured strategyproofness;
5. make all of this explorable in a local browser application.

The reference is Chapter 5 of G. Weiss (ed.), *Multiagent Systems*, 2nd ed., MIT Press, 2013 (printed pp. 181–185 and 199–201). ASCII names $a_1, a_2, \ldots$ stand for the book's indexed arguments.

2 Background

2.1 Abstract argumentation frameworks

An argumentation framework is a pair $AF = (A, R)$ where A is a finite set of arguments and $R \subseteq A \times A$ is the attack relation; $(a, b) \in R$ is written $a \rightharpoonup b$. Self-attacks, mutual attacks and arbitrary directed cycles are all permitted. For $S \subseteq A$ let $S^+ = \{b \mid \exists a \in S : \ a \rightharpoonup b\}$ be the set of arguments attacked by S.

- S is *conflict-free* iff $S \cap S^+ = \emptyset$.
- S *defends* a iff every attacker of a lies in S^+. An unattacked argument is defended by every set, including $\emptyset$.
- The *characteristic function* is $F(S) = \{a \in A \mid S \text{ defends } a\}$. It is monotone, and $F(S)$ may contain arguments outside S.

Table 1 lists the semantics implemented. Preferred and semi-stable use *set inclusion* for maximality, never cardinality: an incomparable extension is not discarded because another one is larger.

Table 1: Semantics implemented in the engine.

Semantics	Definition
Admissible	S is conflict-free and $S \subseteq F(S)$.
Complete	S is conflict-free and $S = F(S)$.
Grounded	The least fixed point of F, obtained by iterating F from $\emptyset$ until nothing changes. It exists and is unique.
Preferred	An inclusion-maximal complete (equivalently, admissible) set.
Stable	S is conflict-free and $S^+ = A \setminus S$. May not exist (for example on an odd cycle).
Semi-stable	A complete set whose *range* $S \cup S^+$ is inclusion-maximal.

2.2 Labellings and acceptance status

A *labelling* assigns each argument one of in, out or undec. It is *legal* when an argument is in iff all its attackers are out, and out iff at least one attacker is in. Every complete extension S yields exactly one legal labelling: $\mathsf{in} = S$, $\mathsf{out} = S^+$ and $\mathsf{undec} = A \setminus (S \cup S^+)$. The implementation uses this bijection rather than testing all $3^{|A|}$ candidate labellings.

Given a semantics, an argument is *skeptically accepted* if it is in every extension, *credulously accepted* if it is in some but not all, and *rejected* if it is in none. If a semantics has no extension (stable semantics on an odd cycle), acceptance is reported as *undefined* rather than vacuously accepting every argument.

2.3 Strategic revelation

Let the arguments be partitioned into ownership sets $A_1, \ldots, A_m$, one per agent. Agent i may reveal any subset $R_i \subseteq A_i$, including $\emptyset$, but cannot reveal another agent's arguments. The *grounded direct mechanism* takes the revealed arguments $\bigcup_i R_i$, induces the framework on them, and outputs its grounded extension $G(R_1, \ldots, R_m)$. The utility of agent i is

$$u_i(R_1, \ldots, R_m) = \left| G(R_1, \ldots, R_m) \cap A_i \right|,$$

the number of *its own* arguments that are accepted, not the global number of accepted arguments.

Truthful revelation is a *dominant strategy* for agent i iff for every report profile R_{-i} of the others and every $R_i \subseteq A_i$ we have $u_i(A_i, R_{-i}) \geq u_i(R_i, R_{-i})$. The mechanism is *strategyproof* if this holds for every agent (Theorem 5.3 of the book). Theorem 5.4 gives a structural *sufficient* condition: no agent owns two arguments connected by a directed odd-length path in the full framework. The failure of that condition does not by itself imply manipulability.

3 Algorithms

3.1 Exact enumeration of semantics

For frameworks of at most 12 arguments, the engine enumerates all $2^{|A|}$ subsets once, keeps the conflict-free ones, and derives the semantics from them. Complete extensions are the conflict-free fixed points of F; preferred, stable and semi-stable extensions are filtered from them by inclusion-maximality, range-maximality and the stable condition. The grounded extension is computed by direct iteration, which also yields the *trace* $\emptyset \to F(\emptyset) \to F^2(\emptyset) \to \ldots$ shown in the interface.

3.2 Best responses and the strategyproofness check

For an agent i and fixed reports R_{-i} of the others, *best-response enumeration* evaluates all $2^{|A_i|}$ own reports and returns *every* report that attains the maximum utility. Ties matter because truthful reporting need only be a weak best response.

The Theorem 5.3 check enumerates every agent, every combination of the *other* agents' reports (they are not restricted to be truthful) and every own report, and compares each with full revelation. A strictly better alternative disproves strategyproofness and is returned as a *witness*: the agent, the others' reports, the truthful and deviating reports, both outcomes and both utilities. If none exists, truthful reporting is weakly dominant for all agents. Writing $n = |A|$ and m for the number of agents, the complete check compares $m \cdot 2^n$ reports against the truthful one. Figure 1 confirms that every strategyproof instance in the experiment used exactly $3 \cdot 2^n$ comparisons.

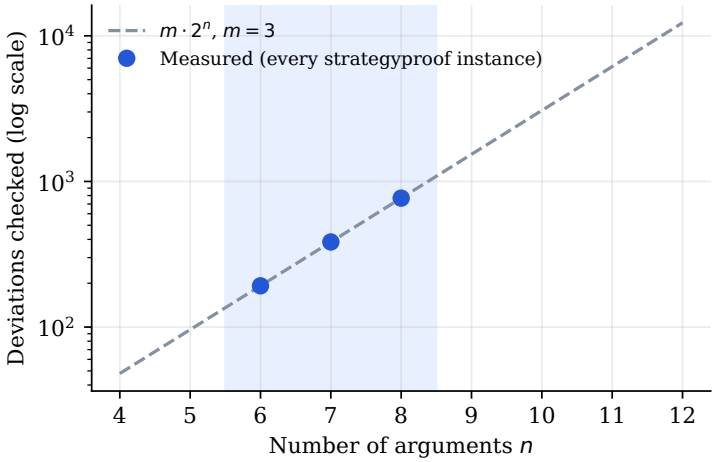


Figure 1: Number of deviations compared by the exhaustive check for strategyproof instances ($m = 3$ agents). Measured values coincide with $m \cdot 2^n$ for $n = 6, 7, 8$; the dashed curve extrapolates to the interface's limit of $n = 12$.

3.3 The odd-walk condition

The structural condition is evaluated separately and never used as a shortcut for the exhaustive check. It asks whether some agent owns arguments a, b such that an odd-length directed path leads from a to b. The implementation adopts the documented *directed-walk* convention: vertices may repeat, and same-endpoint odd cycles count. From each source a breadth-first search explores states $(\text{vertex}, \text{parity})$, flipping the parity on each attack. There are at most $2|A|$ states per source, so cycles terminate, and the total cost is $O\big(n(n + |R|)\big)$. This convention is conservative: under a stricter simple-path reading the condition can hold in more graphs. A failed condition is therefore reported only as a failure of a *sufficient* condition.

4 System design

4.1 Architecture

Figure 2 shows the structure. The domain layer is pure Python and independent of HTTP. A thin adapter layer validates requests, runs use cases and manages background jobs. The

browser frontend is plain HTML, CSS and JavaScript served by the same process; it contains no algorithms.



Figure 2: Layered architecture. Dependencies point left to right; the domain does not know about HTTP or the UI.

Table 2 summarises the responsibilities of each module.

Table 2: Source modules and their responsibilities.

Module	Responsibility
`engine.py`	`ArgumentationFramework`: conflict-freeness, defence, F, admissible/complete/grounded/preferred/stable/semi-stable extensions, legal labellings, acceptance.
`strategic.py`	`GroundedMechanism`: ownership, reports, induced outcome, utilities, best responses, Theorem 5.3 check, Theorem 5.4 condition.
`experiments.py`	Seeded instance generation and the batch runner that writes the saved results.
`examples.py`	Named fixtures (textbook figures and examples) shown in the interface.
`webapp/validation.py`	Parses request JSON into domain objects and rejects malformed input.
`webapp/services.py`	Use cases: `analyse` and `strategic`.
`webapp/jobs.py`	Background experiment jobs: one active job, five retained, polled by the browser.
`webapp/http.py`	HTTP routing, payload and content-type limits, same-origin check on writes, static files; binds to `127.0.0.1`.

4.2 HTTP interface

Table 3: Routes served by the local application.

Route	Purpose
`GET /api/health`	Readiness and version.
`GET /api/examples`	Fixture frameworks with ownership.
`POST /api/analyse`	Extensions, labellings, set checks and acceptance for a framework.
`POST /api/strategic`	Outcome, utilities, strategyproofness, best responses.
`POST /api/experiments`	Start a background experiment; returns a job id.
`GET /api/jobs/<id>`	Poll job progress and result.
`GET /api/saved-experiments`	The submitted 5,000-instance results.

The application has no third-party Python dependency, no external API and needs no internet connection. It is intended for a trusted local workstation: payload size, graph size and experiment

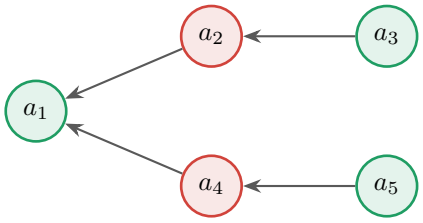
size are bounded, and write routes reject cross-origin browser requests.

4.3 User interface

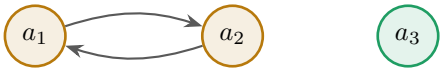
The interface has three workspaces, each fitting on one screen on a desktop display; on narrow screens the sections stack. The *Framework studio* combines an editor, an interactive attack graph coloured by the grounded labelling, a quick check of an arbitrary set S, and tabs for extensions, acceptance status and labellings. The *Strategic playground* shows each agent's arguments as toggles for hiding or revealing, and presents the utilities, both theorem verdicts and the full best-response table. The *Experiments* workspace runs or loads experiments and shows summaries and individual instances.

5 Validation on the textbook examples

Every example below is a fixture in the application and an assertion in the test suite. In the figures, green nodes are in (grounded), red nodes are out and amber nodes are undec.



(a) Figure 5.1: grounded $= \{a_1, a_3, a_5\}$.

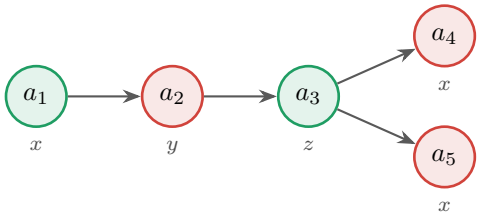


(b) Figure 5.2: grounded $= \{a_3\}$.

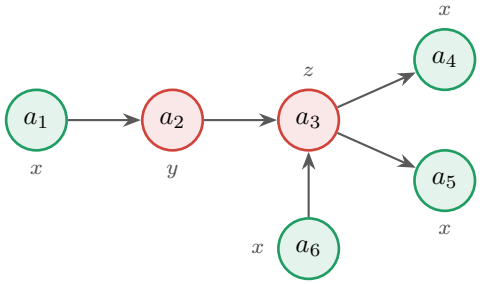
Figure 3: Abstract frameworks from the textbook (Examples 5.5–5.8).

Figure 5.1 (Examples 5.5, 5.6, 5.8). The attacks are $a_2 \rightharpoonup a_1$, $a_4 \rightharpoonup a_1$, $a_3 \rightharpoonup a_2$, $a_5 \rightharpoonup a_4$. The set $\{a_3, a_5\}$ is conflict-free and admissible, and it defends a_1, so it is not complete. Grounded iteration gives $\emptyset \to \{a_3, a_5\} \to \{a_1, a_3, a_5\}$, and this final set is the unique complete, preferred, stable and semi-stable extension.

Figure 5.2 (Examples 5.7, 5.8). a_1 and a_2 attack each other and a_3 is unattacked. The complete extensions are $\{a_3\}$, $\{a_1, a_3\}$ and $\{a_2, a_3\}$; the grounded extension is $\{a_3\}$; the preferred and stable extensions are $\{a_1, a_3\}$ and $\{a_2, a_3\}$. There are three legal labellings. Under preferred semantics a_3 is skeptically accepted and a_1, a_2 are credulously accepted.



(a) Example 5.10 (full reports).



(b) Example 5.11 (full reports).

Figure 4: The strategic examples. Letters below nodes give the owning agent; colouring shows the grounded outcome of the fully revealed framework.

Example 5.10. Agent x owns $\{a_1, a_4, a_5\}$, y owns $\{a_2\}$ and z owns $\{a_3\}$. With full reports the grounded outcome is $\{a_1, a_3\}$ and the utilities are $(1, 0, 1)$. If x hides a_1, the outcome becomes $\{a_2, a_4, a_5\}$ and x's utility rises to 2: a strictly profitable deviation. *Erratum:* the book's utility sentence swaps y and z; the implementation and tests use the values above.

Example 5.11. Adding a_6, owned by x, with $a_6 \rightharpoonup a_3$ gives the full-report outcome $\{a_1, a_4, a_5, a_6\}$ and utility 4 for x. The exhaustive check verifies truthfulness against *every* profile of the others, not just this one. The odd-walk condition nevertheless *fails* because of $a_1 \rightharpoonup a_2 \rightharpoonup a_3 \rightharpoonup a_4$. So this is a strategyproof framework in which the sufficient condition does not hold. *Erratum:* the argument that neutralises the indirect self-defeat is a_6, not a_5 as written in the discussion in the book.

Further fixtures. The tests also cover a defence chain, mutual attack, an odd cycle (no stable extension, all arguments undecided; Figure 5), self-attacks, the empty framework, agents with no arguments, and harmless internal conflict.

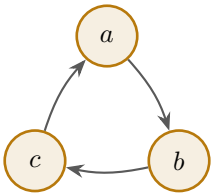


Figure 5: Odd cycle $a \rightharpoonup b \rightharpoonup c \rightharpoonup a$: the only complete extension is $\emptyset$, there is no stable extension, and stable acceptance is reported as undefined.

6 Experimental method

The submitted experiment samples 1,000 instances for each of five attack probabilities $p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, in total 5,000 instances, with Python's seeded generator (seed 2026). The assignment text omitted the probability values; these are explicit provisional choices and are stored with the results.

- The number of arguments n is uniform on $\{6, 7, 8\}$.
- Each ordered pair of distinct arguments is an attack independently with probability p. Directed cycles are allowed; self-attacks are excluded by default (a command-line and browser option enables them).
- Each argument is assigned independently and uniformly to one of three agents, rejecting assignments in which some agent is empty. Ownership is thus uniform over surjective assignments, not over size vectors.
- The random stream continues across probability groups in the stated order.

Every instance runs the exhaustive Theorem 5.3 check and, separately, the Theorem 5.4 condition. Each record keeps the graph, ownership, both flags, the number of comparisons and any witness, so every exceptional instance is identifiable.

To check the generator, Figure 6 compares the measured mean number of attacks with the expectation $p\,\mathbb{E}[n(n-1)] = p \cdot 42.67$; they agree closely.

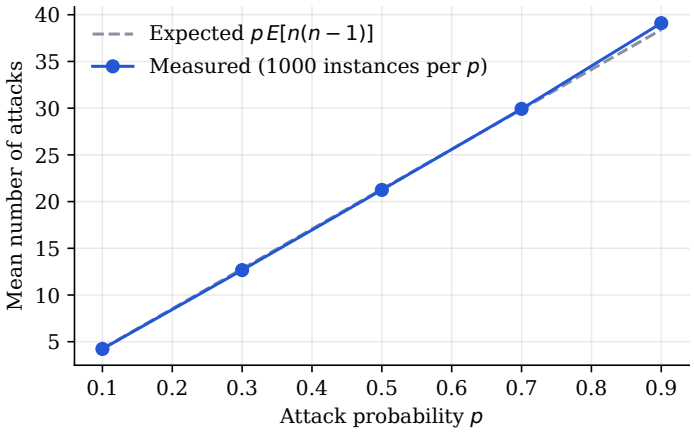


Figure 6: Sanity check of the random-graph generator.

7 Results

7.1 Strategyproofness against attack density

Table 4 and Figure 7 show that strategyproofness becomes rare as the attack graph gets denser: 83.7% of instances at $p = 0.1$, falling to 0.1% at $p = 0.9$. Overall 1,054 of 5,000 instances (21.1%) were strategyproof. Intervals in the table are 95% Wilson score intervals for a binomial proportion of 1,000 trials. This is an empirical statement about the specified generator and seed; it is not a proof of monotonicity.

Table 4: Results by attack probability (1,000 instances each). "SP" means strategyproof.

p	SP	Fraction	95% interval	Cond. holds	Holds, not SP	SP, cond. fails
0.1	837	0.837	[0.813, 0.859]	276	0	561
0.3	188	0.188	[0.165, 0.213]	9	0	179
0.5	24	0.024	[0.016, 0.035]	0	0	24
0.7	4	0.004	[0.002, 0.010]	0	0	4
0.9	1	0.001	[0.000, 0.006]	0	0	1
Total	1054	0.211		285	0	769

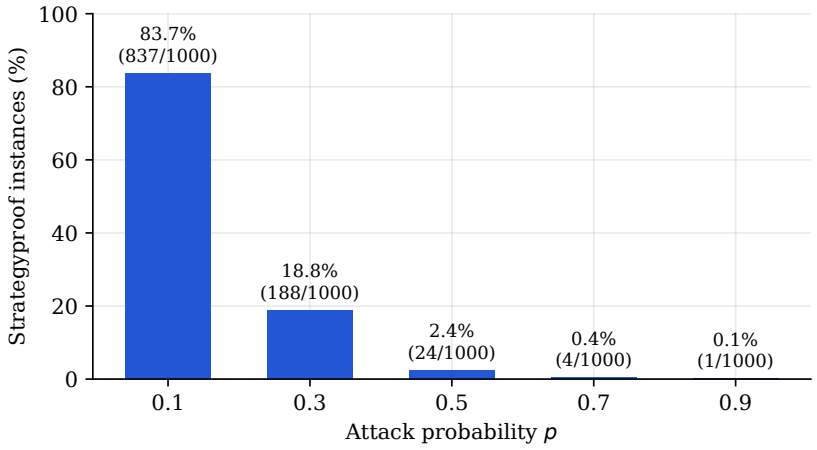


Figure 7: Fraction of strategyproof instances by attack probability.

Splitting by instance size (Figure 8) shows that larger graphs are less often strategyproof at every density: at $p = 0.1$ the fraction falls from 90.7% for $n = 6$ to 74.3% for $n = 8$, and at $p = 0.3$ from 32.9% to 7.7%. More arguments mean more opportunities to hide something profitably.

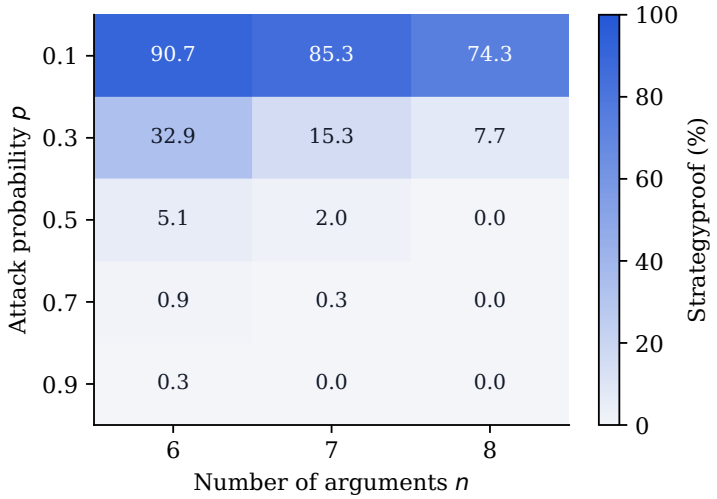


Figure 8: Percentage of strategyproof instances by attack probability and number of arguments.

7.2 The structural condition versus measured strategyproofness

Figure 9 splits each group of 1,000 instances into three classes: the condition holds (green), strategyproof although the condition fails (blue), and manipulable (red).

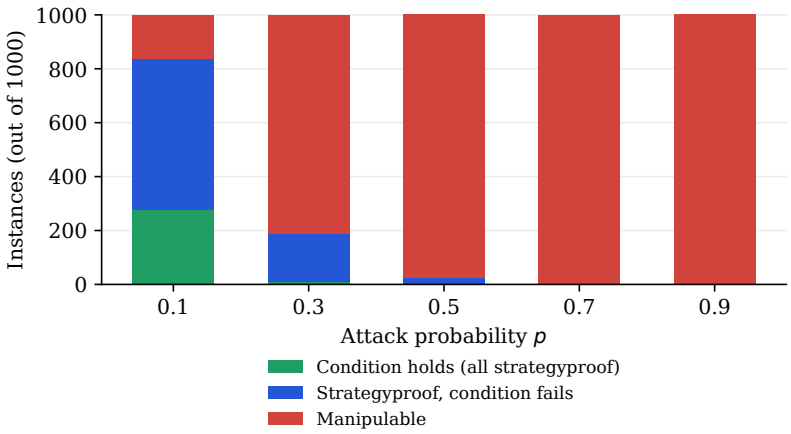


Figure 9: How the sufficient condition relates to measured strategyproofness.

The condition never failed to protect. There are *zero* instances in which the condition holds but strategyproofness fails; the saved list of such counterexamples is empty. All 285 condition-satisfying instances were strategyproof. This supports the theorem on the sampled population; it does not replace its proof, and a nonzero count would have required checking the witness and the cycle convention before claiming a contradiction.

The condition is far from necessary. In 769 instances the mechanism is strategyproof although the condition fails; they are listed in full in `results/converse_examples.json`. In the sample these outnumber the condition-satisfying instances by almost three to one. Their frequency is exactly why a failed condition should not be read as manipulability.

A sparse case: p0.1-trial1. Here $A = \{a, \ldots, f\}$ with attacks $c \rightharpoonup f$, $d \rightharpoonup a$, $d \rightharpoonup c$, $d \rightharpoonup e$, and ownership Agent $1 = \{a\}$, Agent $2 = \{d, e, f\}$, Agent $3 = \{b, c\}$. The condition fails because Agent 2 owns both endpoints of $d \rightharpoonup e$. Yet Agent 2 can never do better than utility 2: under full revelation it gets d and f (argument d blocks c, which protects f), and no report can accept both d and e, so it cannot get all three. Hiding d at best swaps d for e. Agent 1 owns a single argument and cannot gain by hiding it, and Agent 3 cannot influence d by hiding b or c. Strategyproofness is explained without the structural condition.

A dense case: p0.9-trial46. With ownership Agent $1 = \{b, c\}$, Agent $2 = \{a, d, e\}$, Agent $3 = \{f\}$, the attacks leave each of the first two agents with at most one acceptable own argument in any conflict-free outcome, and Agent 3 is a singleton. A parity witness still falsifies the condition. The independent audit enumerates 16, 8 and 32 profiles of the other agents respectively and finds a maximum gain from hiding of 0 for every agent. Textbook Example 5.11 gives a more interpretable reason for the same phenomenon: an additional owned defence cancels the indirect self-defeat.

7.3 Independent audit

The saved results were re-verified by `scripts/audit_results.py`, which imports none of the production solver code. It computes grounded extensions by integer-bitmask propagation, precomputes every induced outcome, and compares each own report with truthful revelation over

all profiles of the others. All 5,000 decisions agree, with zero disagreements. This is stronger than reloading a stored strategyproofness flag.

8 Testing

Table 5: Automated verification.

Suite	What it checks
Python unit tests (33)	Definitions and exact textbook expectations, cycles, self-attacks, empty frameworks and agents, tied best responses, invalid ownership and reports.
Independent oracles	For all $2^9 = 512$ directed three-node graphs (self-attacks included), enumeration of every total in/out/undec labelling is compared with the production complete, preferred, stable, semi-stable and grounded results; a separate propagation oracle checks strategyproofness on the same space.
HTTP and job tests	Routes, input validation, cross-origin rejection, background job lifecycle, failures and recovery, bounded retention, isolation between server instances.
Browser tests (24)	Twelve scenarios on desktop and phone viewports with Playwright: graph editing, keyboard toggling of nodes, textbook sets, acceptance changes, hiding incentives, Example 5.11, error messages, import/export, experiments and downloads, responsive layout.
Result audit	Independent re-verification of all 5,000 saved experiment decisions.

All 33 Python tests and all 24 browser tests pass. A GitHub Actions workflow runs the Python suite and the audit on Linux and Windows, and the browser tests on Linux.

9 Limitations and future work

- The algorithms are exact and exponential. The interface caps frameworks at 12 arguments and experiments use 6–8; no claim of scalability is made. SAT or ASP back-ends would be the natural extension for larger graphs.
- The experiment uses one seed and a finite sample. More seeds and further probability values would allow tighter statements about the decline in Figure 7.
- The probability values p are provisional, as the assignment text omitted them.
- The directed-walk convention for the odd-path condition is conservative; if a simple-path reading is required, the comparison should be repeated (the option exists in the code).
- Only abstract frameworks are covered; the structured-argument and dialogue-game mechanisms elsewhere in the chapter are out of scope.
- The server is for local demonstration and is not suitable for exposure to the internet.

10 Conclusion

Argument Lab computes all the required semantics exactly, reproduces the textbook examples (including two errata), and makes the strategic questions explorable. The experiment shows

that strategyproofness is common in sparse attack graphs and nearly disappears in dense ones, that the sufficient structural condition of Theorem 5.4 never failed to guarantee truthfulness in 5,000 trials, and that it is nevertheless far from necessary: most strategyproof instances violate it. Independent oracles and an independent audit support the correctness of both the engine and the saved results.

Reproducing the results

```
python server.py                                          # start the application
python -m unittest discover -s tests -v                   # Python tests
python -m argumentation.experiments --count 1000 --seed 2026
python scripts/audit_results.py                           # independent audit
python scripts/make_report_figures.py                     # regenerate the plots
```

References

[1] G. Weiss (ed.). *Multiagent Systems*, 2nd edition. MIT Press, 2013. Chapter 5, printed pp. 181–185 and 199–201.

[2] I. Rahwan and K. Larson. Mechanism Design for Abstract Argumentation. In *Proc. AAMAS*, 2008.

[3] P. M. Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995.